

Rapport de Projet : RMI vs Agents Mobiles

Analyse comparative de performance

OUNCHARI Imane - ELLAIK Fadwa

1 Introduction

Ce rapport compare les performances de l'architecture Client-Serveur (RMI) et des Agents Mobiles à travers trois scénarios distincts. L'objectif est d'identifier quand la migration de code (Agent) est préférable à l'appel distant classique (RMI).

2 Analyse des Résultats

2.1 Scénario 1 : Cas Simple (Faible interaction)

Dans ce premier test (5 hôtels), l'agent doit charger son code et migrer.

- RMI (192 ms) : Rapide car la connexion est simple et directe.
- Agent (504 ms) : Plus lent à cause du coût initial de déploiement (overhead) et du chargement des classes ('ClassLoader').

Conclusion : Pour peu de données, RMI est plus efficace.

```
fek2775@dazzler:~/Annee_2/notre-projet$ java agentsmobiles.client.LancerHotel
147.127.133.173 147.127.133.163
Serveur prêt sur le port 5000
[HotelAgent] Je démarre chez le client.
Déplacement de HotelAgent vers 2000
agentsmobiles.common.MoveException: Agent migré - Arrêt local
    at agentsmobiles.common.AgentImpl.move(AgentImpl.java:64)
    at agentsmobiles.client.HotelAgent.main(HotelAgent.java:31)
    at agentsmobiles.common.AgentImpl.run(AgentImpl.java:30)
    at java.base/java.lang.Thread.run(Thread.java:829)
--- RÉSULTAT FINAL ---
Voici les hôtels et leurs numéros :
{Hotel D=0102030068, Hotel A=0102030065, Hotel B=0102030066, Hotel C=0102030067}
TEMPS D'EXÉCUTION AGENT : 504 ms
```

(a) Agent Mobile : 504 ms (Overhead important)

```
java.rmi.client.RMIClientHotel 147.127.133.173
[Client RMI] Récupération de la liste des hôtels...
[Client RMI] Récupération des numéros (Boucle d'appels distants)...
--- RÉSULTAT FINAL RMI ---
{Hotel D=0102030068, Hotel A=0102030065, Hotel B=0102030066, Hotel C=0102030067}
Temps d'exécution : 192 ms
ioi8931@cyclope:~/Téléchargements/projet-donnees-reparties/notre-projet$
```

(b) RMI : 192 ms (Efficace pour requêtes simples)

FIGURE 1 – Comparaison - Cas Simple (5 Hôtels)

2.2 Scénario 2 : Cas Complexe (Forte latence réseau)

Le nombre d'hôtels passe à 26. RMI doit effectuer N appels réseaux successifs, tandis que l'agent se déplace une seule fois et travaille localement.

- RMI (191 ms) : Le temps stagne ou augmente avec la latence réseau cumulée.
- Agent (59 ms) : Une fois "chauffé", l'agent est drastiquement plus rapide. Il élimine la latence réseau en traitant les données sur place.

Conclusion : L'agent surpasse RMI quand les interactions sont nombreuses ("Forwarding").

```
fek2775@dazzler:~/Annee_2/notre-projet$ java agentsmobiles.client.LancerHotel
147.127.133.173 147.127.133.163
Serveur prêt sur le port 5000
[HotelAgent] Je démarre chez le client.
Déplacement de HotelAgent vers 2000
agentsmobiles.common.MoveException: Agent migré - Arrêt local
    at agentsmobiles.common.AgentImpl.move(AgentImpl.java:64)
    at agentsmobiles.client.HotelAgent.main(HotelAgent.java:31)
    at agentsmobiles.common.AgentImpl.run(AgentImpl.java:30)
    at java.base/java.lang.Thread.run(Thread.java:829)
--- RÉSULTAT FINAL ---
Voici les hôtels et leurs numéros :
{Hotel H=0102030072, Hotel I=0102030073, Hotel J=0102030074, Hotel K=0102030075, Hotel L=0102030076, Hotel M=0102030077, Hotel N=0102030078, Hotel O=0102030079, Hotel P=0102030080, Hotel Q=0102030081, Hotel R=0102030082, Hotel S=0102030083, Hotel T=0102030084, Hotel U=0102030085, Hotel V=0102030086, Hotel W=0102030087, Hotel X=0102030088, Hotel Y=0102030089, Hotel Z=0102030090, Hotel A=0102030065, Hotel B=0102030066, Hotel C=0102030067, Hotel D=0102030068, Hotel E=0102030069, Hotel F=0102030070, Hotel G=0102030071}
TEMPS D'EXÉCUTION AGENT : 59 ms
```

(a) Agent Mobile : 59 ms (Très rapide)

```
java.rmi.client.RMIClientHotel 147.127.133.173
[Client RMI] Récupération de la liste des hôtels...
[Client RMI] Récupération des numéros (Boucle d'appels distants)...
--- RÉSULTAT FINAL RMI ---
{Hotel H=0102030072, Hotel I=0102030073, Hotel J=0102030074, Hotel K=0102030075, Hotel L=0102030076, Hotel M=0102030077, Hotel N=0102030078, Hotel O=0102030079, Hotel P=0102030080, Hotel Q=0102030081, Hotel R=0102030082, Hotel S=0102030083, Hotel T=0102030084, Hotel U=0102030085, Hotel V=0102030086, Hotel W=0102030087, Hotel X=0102030088, Hotel Y=0102030089, Hotel Z=0102030090, Hotel A=0102030065, Hotel B=0102030066, Hotel C=0102030067, Hotel D=0102030068, Hotel E=0102030069, Hotel F=0102030070, Hotel G=0102030071}
Temps d'exécution : 191 ms
```

(b) RMI : 191 ms (Latence cumulée)

FIGURE 2 – Comparaison - Cas Complexe (26 Hôtels)

2.3 Scénario 3 : Compression de Fichiers (Bande passante)

Il s'agit de traiter un fichier de 1,9 Mo.

- RMI (216 ms) : Doit télécharger tout le fichier (1,9 Mo) avant de le compresser. Goulot d'étranglement réseau.
- Agent (63 ms) : Migre vers le fichier, le compresse sur le serveur, et ne transfère que le résultat réduit (380 Ko).

Conclusion : L'agent économise la bande passante en déplaçant le traitement vers les données.

```
java agentsmobiles.client.LancerCompress 147.127.133.173
Serveur prêt sur le port 5001
[CompressAgent] Je pars chercher et compresser le fichier.
Déplacement de AgentZip vers 4000
agentsmobiles.commun.MoveException: Agent migré - Arrêt local
    at agentsmobiles.commun.AgentImpl.move(AgentImpl.java:64)
    at agentsmobiles.client.CompressAgent.main(CompressAgent.java:25)
    at agentsmobiles.commun.AgentImpl.run(AgentImpl.java:30)
    at java.base/java.lang.Thread.run(Thread.java:829)
--- RÉSULTAT FINAL ---
Taille originale sur le serveur : 1900000
Taille reçue (après compression 20%) : 380000
TEMPS D'EXECUTION AGENT : 63 ms
```

(a) Agent : 63 ms (Compression distante)

```
[Client RMI] Téléchargement du fichier complet...
[Client RMI] Fichier reçu (1900000 octets). Compression locale...
--- RÉSULTAT FINAL RMI ---
Taille finale : 380000
Temps d'exécution : 216 ms
```

(b) RMI : 216 ms (Transfert inutile)

FIGURE 3 – Comparaison - Compression (1,9 Mo)

3 Conclusion Générale

Les résultats expérimentaux confirment la théorie :

1. RMI est idéal pour les architectures stables et les échanges de données faibles.
2. Les Agents Mobiles deviennent performants (gain de 3x à 4x ici) lorsque le coût de communication (latence ou volume) dépasse le coût de déploiement de l'agent. Ils permettent de réduire le trafic réseau et de déporter la charge de calcul.